

Управление на качеството - Тестване на софтуер

Ивайло Андреев + gemini 3 Flash

17 май 2026

Съдържание

1	Осигуряване на качеството (Quality Assurance)	2
1.1	Качество на софтуера и дефиниции	2
1.2	Алтернативи (стратегии) за осигуряване на качеството	2
2	Тестови дейности, управление и автоматизация	3
2.1	Основни тестови дейности	3
2.2	Управление на тестовите екипи (Модели на организация)	4
2.3	Автоматизация на тестването	4
3	Видове тестване	5
3.1	Тестване с контролни списъци (Checklists)	5
3.2	Тестване с класове на еквивалентност	5
3.3	Разделяне на входния домейн и тестване на границите	6
4	Нива на тестване и приложение на техниките	7
4.1	Специфични видове тестване (преди и след Release)	8
5	Измерване. Метрики – характеристики и класификация	8
5.1	Основни дефиниции	8
5.2	Нива на измерване (Типове скали)	8
5.3	Процес на измерване и характеристики на добрите метрики	9
5.4	Класификация на метриците	9
5.4.1	Метрики за качество на софтуерен ПРОДУКТ (Product Metrics)	10
5.4.2	Метрики за качество на софтуерен ПРОЦЕС (Process Metrics) .	10

1 Осигуряване на качеството (Quality Assurance)

1.1 Качество на софтуера и дефиниции

Осигуряването на качеството (*Quality Assurance* - QA) представлява систематичен модел от всички дейности, нужни за подsigуряване на адекватна увереност, че софтуерният продукт отговаря на определените технически и функционални изисквания (според дефиницията на IEEE). Това е набор от дейности, проектирани изцяло с цел оценка и подобрене на **процеса**, чрез който се създават и поддържат продуктите.

Качество на софтуера се дефинира по два основни начина:

- Степента, до която дадена система, компонент или софтуерен процес отговаря на изрично специфицираните изисквания.
- Степента, до която даден продукт удовлетворява реалните нужди, очаквания и изисквания на клиента или крайните потребители.

Според международния стандарт **ISO** (напр. ISO/IEC 25010), основните характеристики на качеството на софтуерния продукт включват:

1. **Функционалност (Functional Suitability):** Способността на софтуера да изпълнява задачите, отговарящи на заявените нужди.
2. **Надеждност (Reliability):** Степента, до която системата изпълнява дефинираните функции при определени условия за определен период от време.
3. **Използваемост (Usability):** Степента, до която продуктът може да бъде използван от потребители за постигане на целите им с ефективност и удовлетвореност.
4. **Ефикасност (Performance Efficiency):** Производителност спрямо количеството използвани ресурси при определени условия.
5. **Възможност за поддръжка (Maintainability):** Леснота, с която софтуерът може да бъде модифициран, коригиран или подобряван.
6. **Мобилност / Преносимост (Portability):** Леснота при прехвърляне на системата от една хардуерна, софтуерна или оперативна среда в друга.

1.2 Алтернативи (стратегии) за осигуряване на качеството

Инженерингът на качеството разглежда три основни алтернативни подхода (или стратегии) за управление на дефектите:

- **Предотвратяване на дефекти (Defect Prevention):** Премахване на потенциалните източници на грешки, преди те да се проявят в реалния код. Фокусира се върху подобряване на самия процес на разработка (напр. чрез обучения, инспекция на изискванията, добри архитектурни практики).

- **Откриване на дефекти (Defect Detection):** Динамично изпълнение на системата и провеждане на тестови дейности с цел намиране, локализиране и документиране на съществуващи дефекти в програмния код.
- **Ограничаване на дефекти (Defect Containment):** Подход, насочен към ограничаване на обхвата, разпространението и ефекта от дефекти в работеща производствена среда. Целта е да се предотврати критичен срив на цялата система. Примери за това са архитектурните техники *recovery blocks* (блокове за възстановяване) и *N-version programming* (N-версионно програмиране).

2 Тестови дейности, управление и автоматизация

2.1 Основни тестови дейности

Процесът на софтуерно тестване се състои от следните циклични и взаимосвързани фази:

1. **Планиране на тестовете и подготовка:** Дефиниране на тестовата цел (напр. измерване на надеждност, функционално покритие), разпределяне на ресурсите и персонала, избор на формални модели, подходи и техники за тестване.
2. **Конструиране на тестов модел:** Идентифициране на надеждни източници на информация, събиране на необходимите данни, последвано от анализ и формално изграждане на модела на тестване.
3. **Генериране на тестови сценарии (Test Cases) от модела:**
 - **Тестови сценарий (Test Case - TC):** Колекция от входни данни, предварителни условия, стъпки за изпълнение и очаквани резултати, разработени за постигане на определена тестова цел.
 - **Тестова серия (Test Run):** Динамична единица от специфични тестови дейности в общата тестова последователност върху избран обект. Представява еднократно реално изпълнение на един или няколко теста.
4. **Създаване и управление на тестови пакети (Test Suites):** *Test Suite* е логическа колекция от отделни тестови сценарии, които се групират и стартират в определена последователност, докато не се удовлетвори даден критерий за спиране.
5. **Подготовка на тестова процедура:** Дефиниране на точния оперативен ред, технически условия и логика за превключване между различните тестови серии.

6. **Изпълнение на тестовете и наблюдение:** Заделяне на ресурси, реално стартиране на тестовете, събиране на изходните резултати и тяхната верификация чрез **тестови оракули** (механизми за проверка дали реалният изход съвпада с очаквания). Включва и повторно тестване (*Retest*) за потвърждение на отстранените дефекти.
7. **Анализ и проследяване (Тестово измерване):** Анализирание на индивидуални сценарии, измерване на общото тестово покритие, оценка на надеждността на продукта и координиране на дейностите по отстраняване на откритите повреди.

2.2 Управление на тестовите екипи (Модели на организация)

Организационната структура на QA екипите се класифицира в три основни модела:

- **Вертикален модел:** Организацията е изградена изцяло около конкретен софтуерен продукт. Една или повече тестови задачи се асоциират директно с конкретен изпълнител вътре в крос-функционалния продуктов екип.
- **Хоризонтален модел:** Прилага се предимно в големи корпоративни структури. Съществува обособен и силно специализиран централен тестов екип, който отговаря за изпълнението на точно определен тип тестване (напр. само Performance или само Security) за абсолютно всички продукти в компанията.
- **Смесен модел:** Балансиран подход, при който:
 - Ниските нива на тестване (unit, компонентно) се изпълняват локално от екипите, отговорни за съответните проекти.
 - Системното тестване се споделя или ротира между сходни/интегрирани проекти.
 - Осигурява се обща поддръжка и стандартизация на процесите чрез централна QA единица.

2.3 Автоматизация на тестването

Основната цел на автоматизацията е освобождаването на QA инженерите от монотонни, досадни и времеемки повтарящи се задачи, като по този начин се повишава общата продуктивност и тестово покритие.

Възможностите за автоматизация се разделят спрямо трудността и естеството на дейността:

Таблица 1: Възможности за автоматизация в тестовия процес

Ниво на възможност	Тестови дейности и задачи
Висока	Изпълнение на тестове, генериране на тестови данни, анализ на надеждността, анализ на тестовото покритие.
Средна	Подготовка на тестови сценарии, създаване на формални модели.
Ниска	Цялостно планиране на тестовия процес, планиране на тестова процедура, стратегически действия за подобряване на софтуерния продукт.

3 Видове тестване

3.1 Тестване с контролни списъци (Checklists)

Това е систематична еволюция на свободното *Ad-hoc тестване* (при което софтуерът се изпробва без предварителен план, разчитайки на интуицията на тестера). Тестването с контролни списъци въвежда неформални *TODO* структури за организирано проследяване на проверените елементи.

Различават се следните основни типове контролни списъци:

- **Базови:** Прости, линейни списъци от елементи или функционалности, които задължително трябва да се изтестват.
- **Йерархични:** Списъци, структурирани на нива на детайлност. Елементите от по-горните нива съдържат детайлизирани подсписъци от по-ниски нива.
- **Комбинирани (многомерни):** Многомерни структури, при които всеки списък се обхожда и комбинира с елементите от останалите контролни списъци (напр. тестване на списък от роли спрямо списък от операции).
- **Смесени:** Комплексно съчетание между йерархични и комбинирани списъци.

Основни проблеми и ограничения: Трудности при пълното и детайлно покритие на всички функционалности от различни гледни точки; припокриване (дублиране) на елементи в различните списъци; невъзможност за адекватно описване на сложни динамични взаимодействия между компонентите.

3.2 Тестване с класове на еквивалентност

Тази техника се базира на фундаментален математически принцип от дискретната математика: класовете на еквивалентност, породени от релация на еквивалентност

$\equiv_A$ над множество A , представляват **разбиване** на това множество на фамилия от подмножества $C_1, C_2, \dots, C_n$, за които е изпълнено:

$$\forall i, j (i \neq j \Rightarrow C_i \cap C_j = \emptyset), \quad \bigcup_{i=1}^n C_i = A, \quad \text{и} \quad C_i = [x]_A = \{a \in A \mid x \equiv_A a\}$$

В софтуерното инженерство това означава, че входните данни се разделят на групи (класове), за които се предполага, че софтуерът ще се държи по напълно идентичен начин. Ако един тестов сценарий от даден клас премине успешно (или се провали), се счита, че това е валидно за всички останали представители на този клас.

Оперативни стъпки: 1. Дефиниране на класовете на еквивалентност (валидни и невалидни). 2. Избор на точно един представител (тестов сценарий) от всеки клас. 3. Изпълнение с цел постигане на пълно покритие на класовете.

Типове разделяне: Базирано на софтуерни елементи; базирано на определени свойства, релации или логически условия; комбинирано. *Забележка:* Дърветата и таблиците за вземане на решения се разглеждат като йерархични контролни списъци и се прилагат при тестване, базирано на решения/предикати.

3.3 Разделяне на входния домейн и тестване на границите

Метод за генериране на функционални тестови сценарии посредством системен избор на специфични стойности за входните променливи въз основа на структурен анализ на входното пространство. Характеризира се с проверка на входно-изходните зависимости (без задължително експлицитно специфициране на изходите).

Основни стъпки в процеса:

1. Идентифициране на входното пространство и дефиниране на входния домейн.
2. Разбиване на входния домейн на неoverlapping поддомейни.
3. Детайлен анализ на поддомейните с цел дефиниране на техните точни граници по всички измерения.
4. Избор на тестови точки (сценарии), които покриват както вътрешността на поддомейните, така и техните граници.
5. Изпълнение на тестовете, откриване и анализ на резултатите.

Типични проблеми при анализиране на входния домейн:

- **Неопределеност на даден вход:** Софтуерът не притежава логика за обработка на определени входни стойности.
- **Противоречивост за даден вход:** Възникване на срыв или генериране на коренно различни изходи при подаване на абсолютно еднакви входни данни.

Проблеми при определяне на границите на поддомейните:

- **Проблем със затвореността на границите:** Когато една отворена граница погрешно се специфицира или имплементира като затворена (и обратно).
- **Изместване на границата:** Граничната стойност е изместена наляво или надясно в числовото пространство.
- **Липсваща граница:** Границата въобще не е заложена в програмната логика.
- **Излишна граница:** Изкуствено създадена граница, която не съществува в бизнес изискванията.

4 Нива на тестване и приложение на техниките

Софтуерното тестване е строго йерархичен процес, разделен на пет основни нива (подфази):

1. **Тестване на ниво програмна единица (Unit testing):** Фокусира се върху най-малките изолирани програмни елементи (функция, метод, единичен клас). Използва изцяло техниката на **Бялата кутия** (*White-box* - тестване със знание за вътрешната структура на кода). Изпълнява се основно от самите програмисти.
2. **Компонентно тестване (Component testing):** Провежда се над цели софтуерни компоненти, модули или библиотечни пакети. Позволява прилагането както на техники на **Бялата кутия**, така и на **Черната кутия** (*Black-box* - функционално тестване само по вход и изход).
3. **Интеграционно тестване (Integration testing):** Тества интерфейсите, комуникацията и съвместната работа между отделните компоненти при тяхното събиране. Прилага се предимно методът на Бялата кутия, а управлението на тестовото изпълнение се моделира чрез **Машина на крайните състояния** (Краен автомат).
4. **Системно тестване (System testing):** Оценява външните системни операции на цялостния интегриран продукт от гледна точка на крайния потребител. Използва изцяло метода на **Черната кутия**. Моделира се чрез **Крайни автомати** и **Вериги на Марков**.
5. **Тестване за приемане на системата (Acceptance testing):** Финална проверка на готовността на софтуера за предаване на клиента и внедряване в реална експлоатация. При гъвкави методологии (Scrum/Kanban) се провежда в края на всеки спринт. Разчита на статистическо тестване, базирано на реална употреба и **потребителски профили на Муса и Марков**.

4.1 Специфични видове тестване (преди и след Release)

- **Тестване, базирано на дефекти:** Стратегия, използваща исторически или прогнозиран данни за дефекти. Включва техники като инжектиране на дефекти (*defect injection*) и мутационно тестване (*mutation testing*).
- **Бета тестване (Beta testing):** Тестване на софтуера в реална среда чрез контролирана и ограничена доставка до група крайни реални потребители преди официалния пазарен дебют.
- **Регресионно тестване (Regression testing):** Задължителна проверка при пускане на нови версии, целяща да докаже, че съществуващите функционалности не са нарушени от направените промени или фиксове. Фокусира се върху интеграцията между нови и стари компоненти.
- **Диагностично тестване:** Пресъздаване, изолиране и детайлно диагностициране на конкретни критични проблеми, възникнали и докладвани директно от клиенти от реалната производствена среда.

5 Измерване. Метрики – характеристики и класификация

5.1 Основни дефиниции

- **Измерване:** Систематичен процес, при който в съответствие с предварително дефинирани и ясни правила, на определени характеристики на изследвания софтуерен обект се съпоставят конкретни стойности.
- **Мярка:** Стойността (числова или категорично), съпоставена на дадено измерване, която отразява текущото състояние на изследваната характеристика.
- **Метрика:** Математическа функция, която съпоставя състоянията на дадена софтуерна характеристика с техните съответни мерки.

5.2 Нива на измерване (Типове скали)

Стойностите от измерванията се класифицират в четири основни скали:

1. **Номинална (Nominal) скала:** Изброимо множество от категории или текстови стойности, които се присвояват без никакво количествено съотношение помежду си (напр. тип дефект: *"UI"* *"Логически"* *"База данни"*).
2. **Порядкова (Ordinal) скала:** Наредено множество от категории, при което има ясна зависимост подредба, но разликата между тях не може да се измери числено (напр. сериозност на дефект: *"Нисък"* *"Среден"* *"Критичен"*).

3. **Интервална (Interval) скала:** Числови стойности, при които разликата между всеки две последователни единици е напълно еднаква. Позволява прилагането на стандартни математически и статистически операции.
4. **Относителна (Ratio) скала:** Модификация на интервалната скала, при която задължително съществува ясно дефинирана, естествена абсолютна нула (0), показваща пълното отсъствие на дадената характеристика (напр. *брой редове код (LOC)*, *брой открити бъгове*).

5.3 Процес на измерване и характеристики на добрите метрики

Типичният процес на софтуерно измерване следва следните стъпки:

Формулиране на метрична система → Събиране на данни →
→ Анализирание → Интерпретиране → Обратна връзка

За да бъде една софтуерна метрика ефективна, тя трябва да отговаря на следните изисквания:

- **Уместна (Relevant):** Измерва значим, съществен и нетривиален атрибут на системата или процеса.
- **Валидна (Valid):** Измерва точно и реално този атрибут, за чието изследване е създадена.
- **Надеждна / Еднозначна (Reliable):** Дава напълно идентични резултати при прилагане в еднакви условия.
- **Взаимно изключваща се:** Не измерва атрибути, които вече са покрити от други метрики в системата.
- **Оперативни изисквания:** Обективност (не зависи от субекта, който измерва) и неподатливост на манипулации.

5.4 Класификация на метриките

Метриките се класифицират по следните критерии:

- **Според предназначението/целта:** За оценка на необходимите ресурси, за оценка на производителността на разработчиците, за прогнозиране на надеждността на продукта, или за debug-ване и локализиране на проблеми.
- **Според начина на получаване на информацията:** Регистрационен подход или измервателен подход.
- **Според типа на изследвания обект:** Метрики за продукт, за процес или за проект.

5.4.1 Метрики за качество на софтуерен ПРОДУКТ (Product Metrics)

Оценяват състоянието и характеристиките на самия краен софтуерен артефакт:

- **Плътност на дефектите (Defect Density):** Изчислява се като отношение на броя на откритите дефекти към размера на софтуера (напр. дефекти на 1000 реда код - KLOC).
- **Проблеми, докладвани от потребители (Customer Problems):** Количествен брой на инцидентите и дефектите, открити след предаването на продукта в експлоатация.
- **Удовлетвореност на потребителя (Customer Satisfaction):** Субективна или метризирана оценка на потребителското преживяване.

5.4.2 Метрики за качество на софтуерен ПРОЦЕС (Process Metrics)

Фокусират се върху ефективността на самия производствен цикъл и тестването:

- **Честота на грешките (Error Density / Rate):** Динамика на откриване на грешки в кода по време на различните фази на разработка спрямо времето или размера.
- **Сериозност на грешките (Error Severity):** Средна изчислена сериозност на дефектите, открити в кода по време на процеса на разработка, помагаща за оценка на зрелостта на инженеринговия процес.